

Table of Contents

1. Preface
2. Declaration
3. Tech stack
 - 3.1. Pyspark
 - 3.2. duckDB
 - 3.3. Python 3.13.7
4. Install steps
 - 4.1. Request access
 - 4.2. Configure your paths
 - 4.3. Create your virtual environment
 - 4.4. Run requirements.txt
 - 4.5. Run pipeline
 - 4.6. To connect duckDB
5. High level data flow diagram
6. Congif.init
7. Data lineage
8. Assumption
9. Solution features
10. Design Decisions or Trade-Offs
11. Out of scope
12. Bronze layer details
 - 12.1. bronze_articles
 - 12.2. bronze_company_metadata
13. Silver layer details
 - 13.1. Messy Data - date and Revenue
 - 13.2. Company_name matching
 - 13.3. dim_category
 - 13.4. dim_company
 - 13.5. dim_revenue_range
 - 13.6. _build_silver_articles
 - 13.7. silver_articles
14. Gold layer details
 - 14.1. gold_fact_arr_observation
15. Export Files
16. Example Queries

Preface

This document describes a solution for ingesting company data for an interview assignment. It outlines the steps required to install and configure the environment for running the data pipeline.

The solution utilizes a Medallion architecture, powered by PySpark and DuckDB, and incorporates data quality checks at multiple levels.

Declaration

The solution has undergone unit testing and is intended to demonstrate my approach to designing and structuring an end-to-end data system, with emphasis on architecture, data modeling, scalability, lineage, and reliability, not simply the implementation of code.

Given the limited timeframe, I used AI coding tools to accelerate the implementation and focus more time on the system design and engineering decisions. While there may be some implementation issues that can be further refined, the overall solution demonstrates the intended architecture, approach and trade-offs.

Tech stack

1. Pyspark
2. duckDB
3. Python 3.13.7

Install steps

1. Request access to Git repository or unpack the solution yipit_chitwanhumad-main.zip
2. Configure your paths in config/config.ini
3. Create your virtual environment (venv)
 - a. `python -m venv venv`
 - b. `source venv/bin/activate`
4. Run requirements.txt

```
python3 -m pip install -r requirements.txt
```
5. Run pipeline

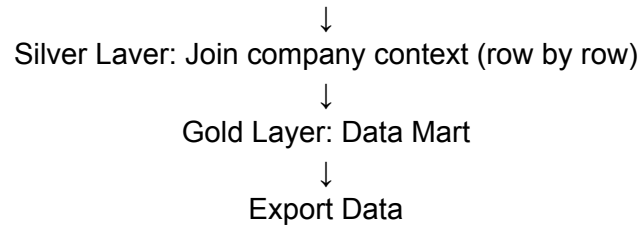
```
python -m src.main
```
6. To connect duckDB locally

```
duckdb /Users/chitwan.humad/yipit/warehouse.duckdb
```

High level data flow diagram

For simplicity all tables are in one schema, the production grade system shall have different schemas.

Bronze Layer : Ingest the Messy external data as is
↓
Silver → _build Layer: Clean + normalize (row by row)



Congif.init

/config/config.init

None

```
[spark]
app_name = yipit_chitwanhumad
master = local[*]
log_level = WARN
warehouse_dir = spark-warehouse

[paths]
PROJECT_ROOT = /Users/chitwan.humad/yipit # change it. <>/yipit
source_data = data/in # no need to change
output_data = data/out # no need to change
duckdb_path = /Users/chitwan.humad/yipit/warehouse.duckdb #change
it. <>/yipit
```

Data lineage

1. Bronze - File meta data like `file_name` and `file_timestamp`
2. Silver - `article_id` + `is_current` + `inserted_at`

Assumption

1. Column `article_id` is coming uniquely from the source system

Solution features

1. Read csv and json files
2. Main file and data lineage
3. Process a file only once (error if the same file processes again)
4. Record files metadata

5. Schema evaluation
6. Normalization
7. Data cleaning
8. Time-series
9. Error handling for use cases

Design Decisions or Trade-Offs

1. **Pandas vs. PySpark**

We used Pandas because the data is small and local. This is fast and easy to test. But if data grows very large, we will need to rewrite the code in PySpark, since Pandas code does not work well on big data.

2. **Bronze layer keeps full history**

Bronze data is never changed or deleted. Each load adds new rows with `batch_id`, `insert_datetime`, and `file_name`. This gives us full history and lets us go back to the original file anytime for audit. For company meta data, we use `is_current` flag to take the latest information.

3. **Incremental load for Silver**

Silver only processes new or changed data, not everything again. We take the latest `batch_id` records to process, match records using `article_id`. New articles are added with the `is_current` flag. If an old article gets corrected, we update that row instead of adding a duplicate (merge operation).

4. **Gold layer: mixed refresh strategy**

The fact table (ARR observations) is updated incrementally, so old records stay and new ones are added safely. But summary views, like the latest ARR or quarterly ARR, are fully refreshed each run, since they are simple to rebuild from the fact table.

5. **Idempotency mechanism** - Using file meta data to avoid duplicate runs `file_name`, `file_timestamp` and `insert_datetime`

6. **Data quality / correctness** - Schema Validation, fuzzy-matching precision-vs-recall trade-off

7. Guessing groupings is quick but inconsistent. A data-ops taxonomy takes longer to build, yet keeps results standardized and reliable long-term. It is manual work as a trade off to the better data quality.

Out of scope

1. Orchastration
2. Schedule and alerts via email or slack

Bronze layer details

bronze_articles

Articles (CSV files) → `bronze_articles`

`bronze_articles` loads CSV files from `source_data`, recording lineage via `file_name`, `file_timestamp` (source file's mtime), and `insert_datetime` (load time). Every file processed in a single pipeline run shares one `batch_id`. Before inserting, it checks for an existing `file_name/file_timestamp` match; matching files are skipped, while a re-saved file loads as new instead.

`bronze_company_metadata`

Company metadata (JSON files) → `bronze_company_metadata`

`bronze_company_metadata` loads JSON files from `source_data`, flattening each company-keyed object into rows tagged with `file_name`, `file_timestamp` (source mtime), and `insert_datetime`. A file is skipped entirely if its `file_name/file_timestamp` pair was already loaded. New data for an existing company expires the prior row's `is_current` flag before inserting new version as current.

Where files come from

Any CSV or JSON file placed under the configured `source_data` folder (searched recursively) is picked up automatically on the next run.

Schema requirements

- Articles must include: `article_id`, `title`, `company_name`, `published_date`, `category`, `revenue`, `summary`, `url`, `author`, `word_count`
- Company metadata (JSON) must be an object keyed by company name, and each company must include: `founded_year`, `headquarters`, `employee_count`, `industry`, `is_public`, `stock_ticker`
- If a file doesn't match, the pipeline stops with a clear error rather than loading partial/bad data.

Duplicate protection

Each file is identified by its name + last-modified time. If that exact combination was already loaded, the file is skipped — so re-running the pipeline is safe and won't create duplicate rows. Saving a new version of a file (which updates its timestamp) is treated as a fresh load.

Batch tracking (articles)

Every pipeline run gets one shared batch number. All article files loaded in that run are tagged with it, making it easy to trace "what came in together" or roll back a specific run.

History tracking (company metadata)

Instead of overwriting a company's data when new metadata arrives, the old record is marked as no longer current and the new one is added as the current version. This keeps a full history of how a company's data has changed over time, while still making it easy to query "the latest info" for any company.

Failure behavior

If required files are missing, or a file fails schema checks, the pipeline stops immediately rather than continuing with incomplete data.

Silver layer details

Messy Data - date and Revenue

Dates

1. Bronze stores the raw string as-is (`bronze_articles.published_date`, still `STRING`).
2. `_date_parser.parse_date()` (dateutil-based) is registered as a **DuckDB scalar function** — `_register_parse_date()` calls `conn.create_function("parse_date", parse_date, return_type="DATE")`.

It's invoked directly in SQL while building the silver staging table:

```
b.published_date AS published_date_as_source
```

3. `parse_date(b.published_date) AS published_date`
So both the raw string and the parsed `DATE` are kept side by side — nothing is discarded, unparseable dates just become `NULL` in the typed column while the source string survives in `published_date_as_source`.
4. One difference from what we drafted here: the repo's version uses `fuzzy=True` (not `False`), so dateutil will also extract a date out of noisier strings that aren't pure dates.
5. A `dim_date` table holds one row per calendar date with derived columns (year, quarter, month, week, day) so daily, monthly, quarterly, or yearly reports simply group/join on it instead of recomputing dates.

Revenue — a longer chain, run in 4 stages

1. **Raw preserved:** `revenue_as_source` keeps the original string, same pattern as dates.
2. **Parse into parts (SQL-side):** `parse_revenue()` wraps `normalize_revenue()` and is registered as a DuckDB function returning a `STRUCT(revenue_currency, revenue_actual, revenue_min_currency, revenue_max_currency)`, called inline in the same staging query (`parse_revenue(b.revenue) AS rev`, unpacked into 4 columns).
3. **Re-parsed (Python-side):** `_update_revenue_parsed()` then re-runs `parse_revenue()` again, in Python this time, over every `is_current = TRUE` row,

and overwrites those same 4 columns via a temp-table **UPDATE**. This looks like it duplicates step 2's work for all current rows — worth checking whether that's intentional (e.g. to also refresh rows outside this batch) or leftover redundancy.

4. **Currency** → **USD**: `_update_revenue_usd()` calls `get_usd_arr()` (`src/bin/_currency_conversion.py`), which multiplies the amount by a rate looked up from `dim_currency_conversion` (`WHERE base_currency = ... AND is_current = TRUE`). USD amounts get a 1:1 pass-through if no explicit USD row exists. Populates `revenue_actual_usd`, `revenue_min_actual_usd`, `revenue_max_actual_usd`.
5. **Bucketing**: `_update_revenue_range_id()` calls `get_revenue_range_id()` (`src/bin/_revenue_range.py`), which now loads the bucket boundaries **dynamically from `dim_revenue_range`** (ordered by `range_order`, `range_id 0` excluded as "Unknown") rather than a hardcoded list — same discrete-vs-midpoint logic we built, just data-driven instead of a Python constant.

Company_name matching

Every article in bronze carries a raw `company_name` string. Matching it to the right row in `dim_company` happens in two passes, and the `company_matched` column records which pass (if either) succeeded — so anyone auditing the data can see exactly how confident a given match is, or spot the ones that failed outright.

company_matched	valueMeaning
Exact	company_name matched a dim_company.company_name value exactly, byte-for-byte, in the initial build.
Fuzzy	No exact match existed, but one of four approximate-matching techniques (below) found a confident candidate.
(the raw company_name itself)	Neither pass found anything. The column is set to the original source string instead of a label, so an unmatched company is immediately visible and searchable — you can filter <code>WHERE company_matched NOT IN ('Exact','Fuzzy')</code> to find every name still needing attention in <code>dim_company</code> .

- **Exact** — the name matched a `dim_company` row exactly.
- **Fuzzy** — no exact match, but a looser check found a confident candidate (ignoring case/legal suffixes like "Inc.", splitting compound names like "A / B", or matching one

name as a substring of the other, but only when both names are at least 4 characters — short strings are too easy to false-match).

- **the raw name itself** — nothing matched at all, so the original string is stored in place of a label, making unmatched companies easy to find and fix in `dim_company`.

Also flagging: upstream value as is in the `company_matched` if could not match at all.

`dim_category`

`dim_category` is a dimension table mapping each category id to its name and a broader, data-ops-defined `category_group` for standardized reporting, with `insert_datetime` tracking when each row was loaded into it.

`dim_company`

`dim_company` is a slowly changing dimension built from `bronze_company_metadata`, holding one current row per `company_name` with core attributes such as industry and headquarters, tracked via `insert_batch_id`, `updated_batch_id`, `inserted_at`, and `updated_at`.

`dim_revenue_range`

This table defines USD revenue buckets with min and max bounds, letting articles' `revenue_actual_usd` be classified consistently into standardized `range_id` groups.

`_build_silver_articles`

`_build_silver_articles` is the silver-layer staging table built from `bronze_articles`: it assigns a **surrogate** `observation_id`, resolves `company_id` and `category_id` via dimension joins, parses `published_date` and revenue (currency, actual, USD-converted, `range_id`) while preserving raw source values, and tracks history through `is_current` flags, expiring prior versions per `article_id`, with batch and timestamp columns. This is a staging table having all bronze layer data lineage.

`silver_articles`

This is the final reporting-ready fact table, rebuilt from scratch each run (`CREATE OR REPLACE TABLE`) from the staging table `_build_silver_articles`, keeping **only current-version rows** (`is_current = TRUE`) and joining in every dimension table.

Column	Source	What it is
--------	--------	------------

observation_id	staging	Surrogate key for the article version
article_id	bronze	Original article identifier
title	bronze	Article title
category_group	dim_category	Standardized category bucket (e.g. "Finance", "AI/ML")
company_matched	derived	'Exact', 'Fuzzy', or 'Not matched' — how company_id was resolved
company_name	derived	The resolved dim_company name if matched; otherwise the raw unmatched source name
revenue_min_usd	derived	revenue_min_actual_usd, falling back to revenue_actual_usd if the source was a discrete value, not a range
revenue_max_usd	derived	Same fallback logic, for the max side
revenue_actual_usd	staging	Discrete revenue in USD (NULL for range values)
revenue_range_name	dim_revenue_range	Human-readable bucket label (e.g. "100M-1B")
industry	dim_company	Raw industry string
founded_year	dim_company	Year the company was founded
company_age	derived	current_year - founded_year
headquarters	dim_company	HQ location
employee_count	dim_company	Headcount
public_private_status	derived	'Public' / 'Private' / 'Unknown' from is_public
stock_ticker	dim_company	Ticker symbol, if any

industry_category	derived	industry + category_group concatenated for cross-cut reporting
published_date	staging	Parsed publish date
company_size_category	derived	'Small' (<10K), 'Medium' (≤30K), 'Large' (>30K) by employee count
year	dim_date	Calendar year of published_date
month	dim_date	Calendar month number
mon	derived	3-letter month abbreviation (e.g. "Jan")
quarter_no	derived	"Q1"–"Q4" string

Gold layer details

gold_fact_arr_observation

gold_fact_arr_observation is a CREATE OR REPLACE copy of silver_articles with one addition: arr_usd, using the reported revenue when discrete, or the average of revenue_min_usd and revenue_max_usd when it's a range, plus a gold_loaded_at timestamp, forming the base table both CSV exports filter from, built via Spark SQL, reloaded into DuckDB.

Export Files

File	Description
_build_silver_articles.csv	staging/transformation data
silver_articles.csv	silver later data
ai_articles_enriched.csv	Gold data
ai_ml_arr_gt_50m_2022_2024.csv	Use case result

Please note: The solution has some additional rows because I have tested the solution with more data and files.

Example Queries

SQL

```
SELECT *
FROM gold_fact_arr_observation
WHERE (
    category_group = 'AI/ML'
    OR lower(COALESCE(category_name, '')) IN (
        'machine learning',
        'ai/ml',
        'ai & ml',
        'artificial intelligence'
    )
    OR lower(COALESCE(industry, '')) IN (
        'ai/ml',
        'machine learning',
        'artificial intelligence'
    )
)
AND year BETWEEN 2022 AND 2024
AND arr_usd IS NOT NULL
AND arr_usd > 50000000
```